Projekt numeriska metoder (SF1546)

Värmeöverföring: Från teori till numerisk lösning

Författare:
Sebastian Alin (20070115-4155, sebali@kth.se)
Robert Falk (20040623-5150, robertef@kth.se)
Pontus Graf von Haslingen (20060907-0073, pgvhgvs@kth.se)

Datum: 2026-04-27

Innehållsförteckning

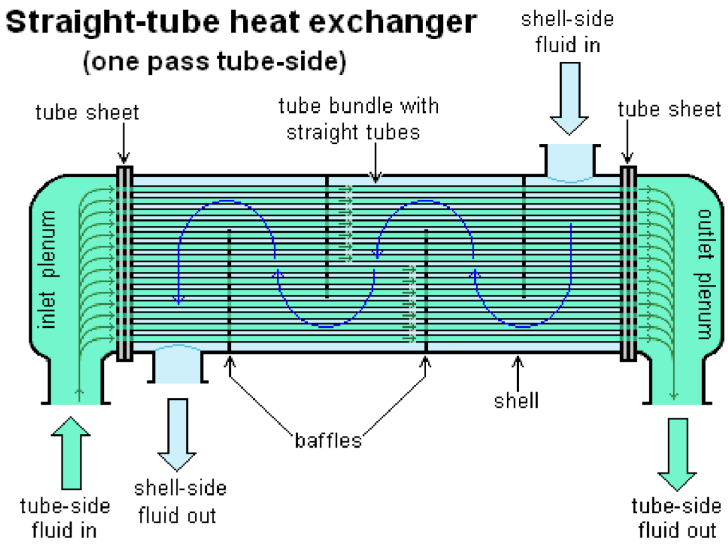
1 Introduktion . . . 3
2 Problemformulering Metod . . . 4
2.1 Värmeväxlare del 1.1 . . . 4
2.2 Värmeväxlare del 1.2 . . . 4
2.3 Derivataapproximationer del 2.1 . . . 5
2.4 Randvärdesproblem del 2.2 . . . 5
3 Genomförande och Resultat . . . 6
3.1 Värmeväxlare del 1.1 . . . 6
3.1.1 Uppgift 1 . . . 6
3.1.2 Uppgift 2 . . . 6
3.1.3 Uppgift 3 . . . 7
3.2 Värmeväxlare del 1.2 . . . 8
3.2.1 Uppgift 4 . . . 8
3.2.2 Uppgift 5 . . . 8
3.3 Derivataapproximationer del 2.1 . . . 10
3.3.1 Uppgift 6 . . . 10
3.4 Randvärdesproblem del 2.2 . . . 11
3.4.1 Uppgift 7 . . . 11
3.4.2 Uppgift 8.1 . . . 12
3.4.3 Uppgift 8.2 . . . 12
3.4.4 Uppgift 8.3 . . . 13
3.4.5 Del 8.4 . . . 15
4 Diskussion . . . 18
4.0.1 Del 1.1 uppgift 3 C) . . . 18
4.0.2 Del 1.2 uppgift 5 . . . 18
4.0.3 Del 2.1 uppgift 6 b) . . . 18
4.0.4 Del 2.2 . . . 18
5 Bilagor . . . 20
5.1 Bilaga A . . . 20
5.2 Bilaga B . . . 20

1 Introduktion

Detta projekt ingår i kursen SF1546 - Projekt numeriska metoder, och syftar till att tillämpa numeriska metoder för att lösa problem inom värmeöverföring i en värmeväxlare och kylflänsar.

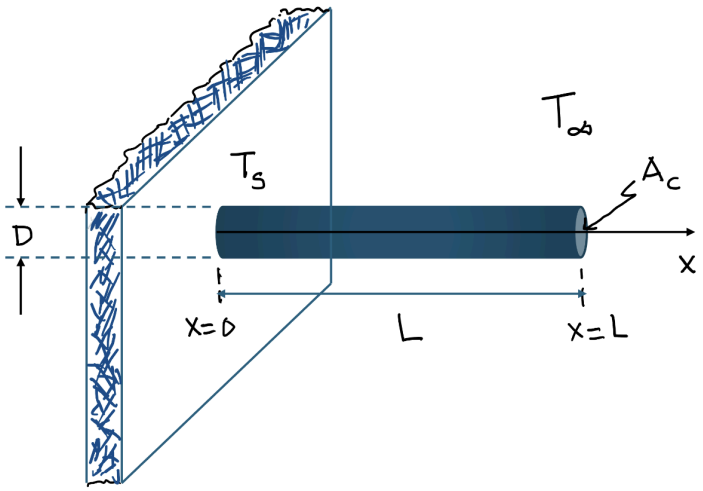
Syftet med projektet är att lösa problem kopplade till verkligheten, med numeriska metoder, särskilt de som innehåller ickelinjära ekvationssystem och randvärdesproblem. Genom att använda dessa metoder, kommer vi att kunna analysera och optimera värmeöverföringen i de givna systemen.

Projektet är indelat i två delar, där den första delen syftar till att lösa ett icke-linjärt ekvationssystem som beskriver värmeöverföringen i en värmeväxlare (Figur 1), med hjälp av Newtons metod ($x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$), för att få beräkna den maximala ytterdiametern $\boldsymbol{d}$ och sedan innerdiameter $\boldsymbol{D_s}$.



Figur 1: Schematiskt diagram över värmeväxlaren och de givna parametrarna (finns även i kursens projektbeskrivning), hämtad från https://commons.wikimedia.org/wiki/File:Straight-tube_heat_exchanger_1-pass.PNG

Den andra delen fokuserar på att lösa ett randvärdesproblem som beskriver värmeöverföringen i en kylfläns (Figur 2). Med hjälp av diskretisering med centrerad differens, och finita differensmetoden, beräknas temperaturfördelningen längs kylflänsen för material med olika egenskaper.



Figur 2: Schematiskt diagram över kylflänsen och de givna parametrarna hämtade från kursens projektbeskrivning.

2 Problemformulering Metod

2.1 Värmeväxlare del 1.1

Värmeväxlarne i det första problemet kan beskrivas av följande icke-linjära ekvationssystem med givna parametrar (Tabell 1):

$$Q = A_{ex} \cdot U_f \cdot \Delta T_m \tag{1}$$

$$U_f = \frac{1}{\frac{d}{D_s \cdot h_t} + d \cdot \frac{R_{fi}}{D_s} + \frac{d \cdot \ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s}} \tag{2}$$

Parameter	Värde	Enhet
R_{fi}	$1.76 \cdot 10^{-4}$	$m^2{}^\circ C/W$
R_{fo}	$1.76 \cdot 10^{-4}$	$m^2{}^\circ C/W$
h_s	356	$W/(m^2{}^\circ C)$
h_t	356	$W/(m^2{}^\circ C)$
k_w	60	$W/(m^\circ C)$

Tabell 1: parametrar för värmeöverföringen i värmeväxlaren

Specifikationer för värmeöverföringen i värmeväxlaren är som följer: $\Delta T_m = 29.6^\circ C$ och $Q = 801368W$; den minsta ytarean som uppfyller specifikationerna är given till $A_{ex} = 64.15m^2$ och skalets innerdiameter är $D_s = 1.219m$.

Alla parametrar förutom rörets ytterdiameter $\boldsymbol{d}$ är kända. Genom att konstruera en funktion $f(d)$ som representerar skillnaden mellan den beräknade värmeöverföringen och den givna värmeöverföringen, kan problemet formuleras som att hitta roten till denna funktion; det vill säga där $f(d) = 0$.

$$f(d) = A_{ex} \cdot U_{f(d)} \cdot \Delta T_m - Q = 0 \tag{3}$$

För att lösa denna rot, användes Newtons metod, som iterativt närmar sig lösningen genom att uppdatera gissningen baserat på funktionen och dess derivata.

$$x_{d+1} = d_n - \frac{f(d_n)}{f'}(d_n) \tag{4}$$

2.2 Värmeväxlare del 1.2

Det andra problemet bygger på del 1.2 (Avsnitt 2.1) med två ytterligare ekvationer, samt givna parametrar (Tabell 2), där tryckskillnaden över värmeväxlaren ΔP måste beaktas. För detta problem är rörets ytterdiameter $\boldsymbol{d}$ och skalets innerdiameter $\boldsymbol{D_s}$ är okända:

$$\Delta P = \frac{c}{D_s^2 (S_t - d)^2} \tag{5}$$

$$A_{ex} = \pi K_1 \frac{D_1^{n_1}}{d^{n_1 - 1}} \tag{6}$$

Parameter	Värde	Enhet
c	0.389	$Ws \cdot m$
S_t	0.016	m
K_1	0.249	m
n_1	2.207	

Tabell 2: Givna parametrar för värmeväxlaren.

Specifikationerna för värmeöverföring är samma som i del 1.1 ($\Delta T_m = 29.6°C$ och $Q = 801368W$), och en given maximal tryckskillnad över värmeväxlaren $\Delta P = 49080Pa$. Genom att använda de två nya ekvationerna, kan både d och D_s lösas samtidigt genom ett system av två icke-linjära ekvationer.

Ekvation 1 kan nu omformuleras till att inkludera D_s som en variabel, där U_f och A_{ex} är funktioner av både d och D_s:

$$Q(d, D_s) = A_{ex}(d, D_s) \cdot U_f(d, D_s) \cdot \Delta T_m \tag{7}$$

$$\Rightarrow F_1(d, D_s) = A_{ex}(d, D_s) \cdot U_f(d, D_s) \cdot \Delta T_m - Q = 0 \tag{8}$$

Ekvation 5 ger en ytterligare ekvation som relaterar d och D_s:

$$F_2(d, D_s) = \frac{c}{D_s^2 (S_t - d)^2} - \Delta P = 0 \tag{9}$$

Dessa funktioner kan tillsamans konstrueras som en vektorvärd funktion: $\vec{F} = (F_1, F_2) = \vec{0}$. Detta system av icke-linjära ekvationer kan sedan lösas med hjälp av Newtons metod för system, där $\vec{x} = (d, D_s)$:

$$\vec{x}_{n+1} = \vec{x}_n - J^{-1}\vec{F}(x_n) \tag{10}$$

2.3 Derivataapproximationer del 2.1

I denna del härleds approximationen i Ekvation 12 med hjälp av Taylorutveckling. Därefter implementera tre olika derivata approximationer i *Python* för $\mathbf{sin(e^x)}$ vid $x = 0.75$.

$$f(x_j) = \frac{-\omega_{j+2} + 4\omega_{j+1} - 3\omega_j}{2h} + \mathcal{O}(h^2) \tag{11}$$

$$f'(x_j) = \frac{3\omega_j - 4\omega_{j-1} + \omega_{j-2}}{2h} + \mathcal{O}(h^2) \tag{12}$$

$$f'(x_j) = \frac{\omega_{j+1} - (\omega_{j-1})}{2h} + \mathcal{O}(h^2) \tag{13}$$

$\omega_j = f(x_j) = f(a + jh)$ där a är startpunkten, b är slutpunkten, och h är steglängden $h = \frac{b-a}{N}$, där N är antalet delintervall.

2.4 Randvärdesproblem del 2.2

Denna del fokuserade på att lösa ett randvärdesproblem som beskriver av differensialekvationen

$$\frac{d^2T}{dx^2} = \alpha_1(T - T_\infty) + \alpha_2(T^4 - T_\infty^4), \qquad x \in [0, L] \tag{14}$$

Samt Dirichlet randvillkor:

$$T(0) = T_s, \qquad T(L) = T_\infty \tag{15}$$

Detta problem kan lösas genom att diskretisera andra ordningens derivata med central differens, där den andra derivatan approximativt kan skrivas som:

$$\frac{d^2T}{dx^2} \approx \frac{T_{j+1} - 2T_j + T_{j-1}}{h^2} \qquad (+\mathcal{O}(h^2)) \tag{16}$$

Där $h = \frac{L}{N}$, där N är antalet delintervall.

Vidare kan uttrycket skrivas för varje givet $T_j = T(hj)$ som en funktion: $F_j(T_{j-1}, T_j, T_{j+1}) = 0$, där $T_0 = T_s$ och $T_N = T_\infty$. Detta ger upphov till ett system av icke-linjära ekvationer ($\vec{F} = (F_1, F_2, ..., F_{N-1}) = \vec{0}$) som kan lösas med hjälp av Newtons metod för system, $\vec{T}^{k+1} = \vec{T}^k - J^{-1}\vec{F}(T^k)$.[1]

3 Genomförande och Resultat

3.1 Värmeväxlare del 1.1

3.1.1 Uppgift 1

I Denna deluppgift formuleras den matematiska modellen för att implementera Newtons metod, genom att beräkna $f(d)$ och dess derivata $f'(d)$ analytiskt. $f'(d)$ kan beräknas genom att använda kedjeregeln och produktregeln för derivering av vilket ger:

$$f'(d) = -A_{ex} \cdot \Delta T_m \cdot U_f{}'(d) \tag{17}$$

$U_f{}'(d)$ kan beräknas genom att använda kvotregeln för derivering, där $g(d)$ är nämnaren i uttrycket för $U_{f(d)}$:

$$g(d) = \frac{d}{D_s \cdot h_t} + d \cdot \frac{R_{fi}}{D_s} + \frac{d \cdot \ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s} \tag{18}$$

$$U_{f(d)} = \frac{1}{g(d)} \Rightarrow U_f{}'(d) = -\frac{g'(d)}{g(d)^2} \tag{19}$$

$g(d)$ är relativt enkelt att derivera bortsätt från den logaritmiska termen, där produktregeln måste tillämpas.[2]

$$\frac{d}{dd}(d \cdot \ln(d/D_s)) = 1 \cdot \ln(d/D_s) + d \cdot \frac{1}{d} = \ln(d/D_s) + 1 \tag{20}$$

$$\Rightarrow g'(d) = \frac{1}{D_s \cdot h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s) + 1}{2k_w} \tag{21}$$

$$f'(d) = -(A_{ex} \cdot \Delta T_m) \cdot \left(-\frac{g'(d)}{g(d)^2}\right) = A_{ex} \cdot \Delta T_m \cdot \frac{g'(d)}{g(d)^2} \tag{22}$$

3.1.2 Uppgift 2

Uppgift 2 syftar till att implementera Newtons metod i *Python*, med den beräknade derivatan, för att iterativt närma sig lösningen för d, med en startgissning $\boldsymbol{d_0 = 0.007m}$ och en tolerens på $\boldsymbol{10^{-8}m}$.

```
def newton_method(d_0, tolerance, max_iter=10000):
    d_n = d_0
    errors = []
    for i in range(max_iter): # Loop until max iterations
        if f_prime(d_n) == 0:  # Avoid division by zero
            print("Derivative is zero. No solution found.")
            return None
        d_next = d_n - f(d_n) / f_prime(d_n) # newtons method formula
        errors.append(abs(d_next - d_n)) # store the error for convergence analysis

        if abs(d_next - d_n) < tolerance:
```

[1] Notera $\overrightarrow{T^k}$ används som superskript och inte som exponent

[2] $\frac{d}{dd}(\ln(d/D_s)) = \frac{d}{dd}(\ln(d) - \ln(D_s)) = \frac{1}{d} - 0$

```
            print(f"Converged to {d_next} after {i+1} iterations.")
            return d_next, errors
        d_n = d_next
    return None
```

Denna implementering ger svaret $\boldsymbol{d = 0.0191314071m \approx 19.1mm}$ efter 4 iterationer.

3.1.3 Uppgift 3

Denna uppgift skall analysera konvergensordningen för Newtons metod genom att använda de beräknade felen från varje iteration, och jämföra lösningen mot pythons egna lösning

a) genom att anta felet som en potensfunktion kan konvergensordningen p uppskattas genom att använda följande formel:

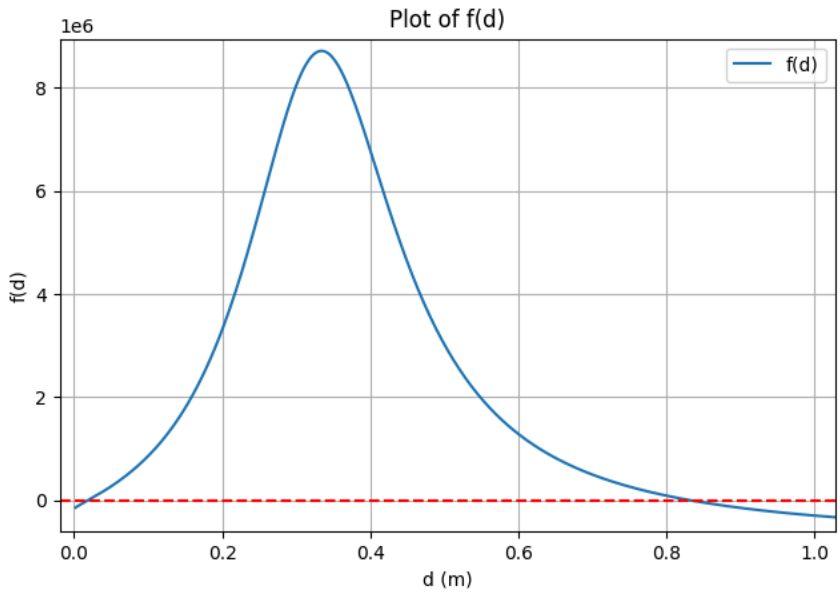
$$e_{k+1} \approx Ce_k^p \Rightarrow e_{k+2} \approx Ce_{k+1}^p \tag{23}$$

$$\frac{e_{k+2}}{e_{k+1}} = \frac{Ce_{k+1}^p}{Ce_k^p} = \left(\frac{e_{k+1}}{e_k}\right)^p \Rightarrow \ln\left(\frac{e_{k+2}}{e_{k+1}}\right) = p\ln\left(\frac{e_{k+1}}{e_k}\right) \Rightarrow p = \frac{\ln(e_{k+2}/e_{k+1})}{\ln(e_{k+1}/e_k)} \tag{24}$$

Implementeringen av detta i *Python* ger nu en uppskattning av konvergensordningen $\boldsymbol{p = 2.3561}$. Det ligger relativt nära den teoretiska konvergensordningen för Newtons metod, som är $\mathbf{2}$.[3]

b) Lösningen från Scipy: `(optimized_solution = optimize.fsolve(f, d_0))` ger $\boldsymbol{d = 0.0191314071m}$, vilket i praktiken är identiskt med vår implementerade lösning, och bekräftar att vår implementation av Newtons metod är korrekt.

c) Genom att plotta funktionen $f(d)$ över intervallet $[0, 2]$ kan vi visuellt bekräfta att det finns två rötter (Figur 3). Den första ligger nära $\boldsymbol{d \approx 0.0191m}$, vilket stämmer med både Newtons metod och SciPy:s fsolve när startgissningen ligger nära denna lösning. Den andra roten ligger nära $\boldsymbol{d \approx 0.8314m}$ (se Avsnitt 4.0.1). Detta stämmer väl med körningarna där startgissningar som 0.4, 0.95 och 1.1 konvergerar mot samma andra rot. För en startgissningen 1.5 konvergerar inte metoden.



Figur 3: plot av $f(d)$ över intervallet $[0, 2]$

[3]Detta kan härledas med Taylor utveckling (se länk), och hämtades från föreläsningsanteckningarna

3.2 Värmeväxlare del 1.2

3.2.1 Uppgift 4

Uppgiften syftar till att formulera det icke-linjära ekvationssystemet så att det kan lösas med Newtons metod för system. I metoden konstruerades en vektorvärd funktion $\vec{F} = (F_1, F_2)$ där F_1 (Ekvation 8) och F_2 (Ekvation 9) är de två ekvationerna som beskriver systemet. För att implementera Newtons metod för system (Ekvation 16), krävs även Jacobimatrisen, som består av de partiella derivatorna av F_1 och F_2 med avseende på variablerna d och D_s. Jacobimatrisen J kan skrivas som:

$$J = \begin{bmatrix} \frac{\partial F_1}{\partial d} & \frac{\partial F_1}{\partial D_s} \\ \frac{\partial F_2}{\partial d} & \frac{\partial F_2}{\partial D_s} \end{bmatrix} \tag{25}$$

$$F_2 = \frac{c}{D_s^2(S_t - d)^2} - \Delta P = \frac{c}{h(d, D_s)} - \Delta P \qquad \left(h(d, D_s) = D_s^2(S_t - d)^2\right) \tag{26}$$

$$\Rightarrow \frac{\partial F_2}{\partial d} = -\frac{c}{h(d, D_s)^2}\frac{\partial h}{\partial d} = -\frac{c(2D_s^2(S_t - d))}{D_s^4(S_t - d)^4} = -\frac{2c}{D_s^2(S_t - d)^3} \tag{27}$$

$$\Rightarrow \frac{\partial F_2}{\partial D_s} = -\frac{c}{h(d, D_s)^2}\frac{\partial h}{\partial D_s} = \frac{c\left(2D_s(S_t - d)^2\right)}{D_s^4(S_t - d)^4} = \frac{2c}{D_s^3(S_t - d)^2} \tag{28}$$

$$F_1 = A_{ex}\frac{d, D_s}{g(d, D_s)} \cdot \Delta T_m - Q, \qquad \left(g(d, D_s) = 1/U_f\right) \tag{29}$$

$$\Rightarrow \frac{\partial F_1}{\partial d} = \Delta T_m \frac{\frac{\partial A_{ex}}{\partial d} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial d}}{g(d, D_s)^2} \tag{30}$$

$$\Rightarrow \frac{\partial F_1}{\partial D_s} = \Delta T_m \frac{\frac{\partial A_{ex}}{\partial D_s} \cdot g(d, D_s) - A_{ex} \cdot \frac{\partial g}{\partial D_s}}{g(d, D_s)^2} \tag{31}$$

$$A_{ex} = \pi K_1 D_1^{n_1} \cdot \left(d^{1-n_1}\right) \Rightarrow \quad \frac{\partial A_{ex}}{\partial d} = -\pi K_1 D_1^{n_1}\frac{n_1 - 1}{d_1^n} \qquad \frac{\partial A_{ex}}{\partial D_s} = \pi K_1 n_1 \frac{D_1^{n_1 - 1}}{d^{n_1 - 1}} \tag{32}$$

$\frac{\partial g}{\partial d}$ har redan beräknats i Ekvation 21

$$\frac{\partial g}{\partial d} = \frac{1}{D_s \cdot h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s) + 1}{2k_w} \tag{33}$$

$$\frac{\partial g}{\partial D_s} = -\frac{d}{D_s^2 \cdot h_t} - d \cdot \frac{R_{fi}}{D_s^2} - \frac{d}{2k_w D_s} \quad \left(\frac{\partial}{\partial D_s}(d \cdot \ln(d/D_s)) = \frac{1}{d/D_s} \cdot -\frac{d}{D_s^2} = -\frac{1}{D_s}\right) \tag{34}$$

3.2.2 Uppgift 5

a) Upgiften syftar till att implementera Newtons metod för system i *Python*, med den beräknade Jacobimatrisen, för att iterativt närma sig lösningen för d och D_s, med startgissningen $d_0 = 0.015m$ och $D_{s0} = 0.8m$, och en tolerens på 10^{-8}; samt analysera konvergensen av lösningen.

```
def jacobian(d, D):
    return np.array([[dF1dd(d, D), dFdD(d, D)],
                     [dF2dd(d, D), dF2dD(d, D)]])

def newton_method_system(d_0, D_0, tolerance, max_iter=10000):
```

```
    d_n = d_0
    D_n = D_0
    errors = []
    for i in range(max_iter):
        F_n = F(d_n, D_n)
        J_n = jacobian(d_n, D_n)
        if np.linalg.det(J_n) == 0:  # Avoid singular Jacobian
            print("Jacobian is singular. No solution found.")
            return None

        delta = np.linalg.solve(J_n, -F_n) # Solve J_n * delta = -F_n for this step
        d_next = d_n + delta[0]
        D_next = D_n + delta[1]
        errors.append(np.linalg.norm(delta))

        if np.linalg.norm(delta) < tolerance:
            print(f"Converged to (d={d_next}, D={D_next}) after {i+1} iterations.")
            return d_next, D_next, errors
        d_n, D_n = d_next, D_next

    print("Maximum iterations reached. No solution found.")
    return None
```

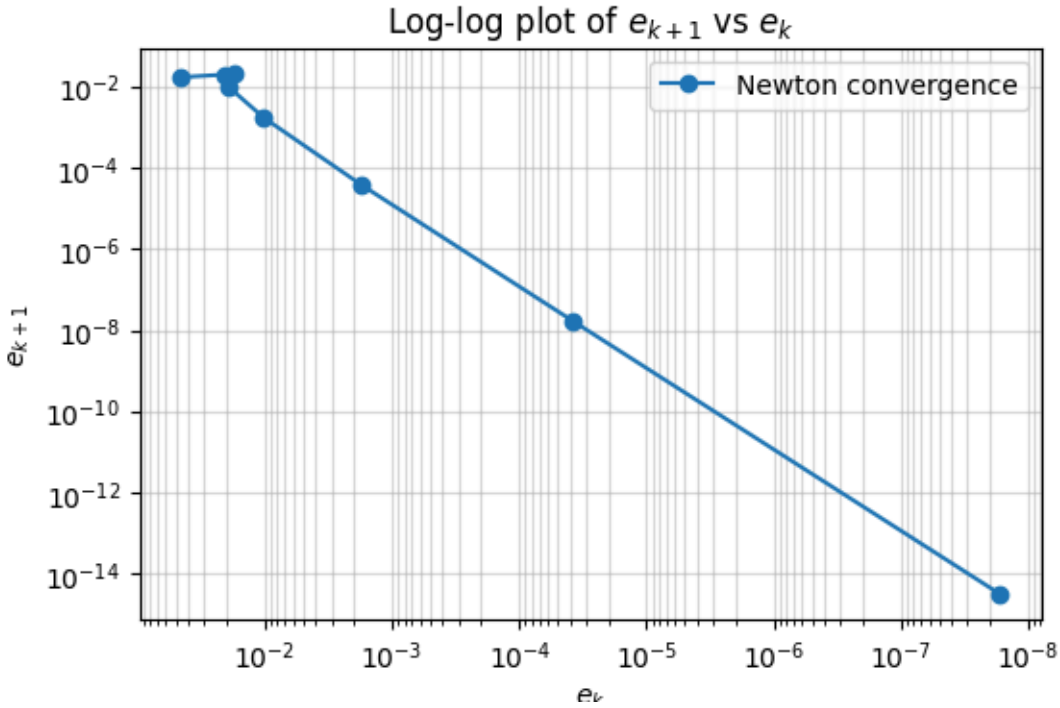
Lösningen i python är mycket lika lösningen från del 1.1 (Avsnitt 2.1), med skillnaden att derivatan är utbytt mot Jacobimatrisen, och att både d och D_s löses ut med hjälp av linjära algebra. Lösningen konvergerar till $\boldsymbol{d = 0.0118869m}$ och $\boldsymbol{D_s = 0.684476m}$ efter 9 iterationer.

b) Genom att använda beräkna felet till normen av uppdateringssteget δ, kan konvergensen analyseras på samma sätt som i del 1.1, och uppskatta konvergensordningen p med Ekvation 24

```
for i in range(2, len(e)):
    order_of_convergence = np.log(e[i]/e[i-1]) / np.log(e[i-1]/e[i-2])
```

```
Order of convergence at iteration 2: -0.18984893081596307
Order of convergence at iteration 3: -0.33340935241195935
Order of convergence at iteration 4: 10.356263334193248
Order of convergence at iteration 5: 2.745105076264547
Order of convergence at iteration 6: 2.1575607548371583
Order of convergence at iteration 7: 2.017792532754369
Order of convergence at iteration 8: 2.0036658242583423
```

Detta ger en konvergensordning som i början svänger kraftigt, men som sedan stabiliseras runt **2**, vilket ilusterras tydligt i Figur 4 och är i linje med den teoretiska konvergensordningen för Newtons metod (se Avsnitt 4.0.2).



Figur 4: Log log plot av e_{k+1} versuse_k (m)

3.3 Derivataapproximationer del 2.1

3.3.1 Uppgift 6

a) Denna uppgift skall härleda den givna approximationen i Ekvation 12 genom att använda Taylorutveckling av funktionen f runt punkten $x+h$, och ta fram det specifika uttrycket för felet i approximationen, som är av ordning h^2.

$$f'(x_j) = \frac{-\omega_{j+2} + 4\omega_{j+1} - 3\omega_j}{2h} + \mathcal{O}(h^2) = \frac{-f(x_j+2h) + 4f(x_j+h) - 3f(x_j)}{2h} \tag{35}$$

$$f(x_j+2h) = f(x_j) + 2hf'(x_j) + \frac{(2h)^2}{2}f''(x_j) + \frac{(2h)^3}{6}f'''(\xi_1) \tag{36}$$

$$f(x_j+h) = f(x_j) + hf'(x_j) + \frac{h^2}{2}f''(x_j) + \frac{h^3}{6}f'''(\xi_2) \tag{37}$$

Genom att substituera dessa uttryck i approximationen

$$\frac{-f(x_j) - 2hf'(x_j) - 2h^2f''(x_j) - 8\frac{h^3}{6}f'''(\xi_1) + 4f(x_j) + 4hf'(x_j) + 2h^2f''(x_j) + 4\frac{h^3}{6}f'''(\xi_2) - 3f(x_j)}{2h} \tag{38}$$

$$= \frac{2hf'(x_j) + 4\frac{h^3}{6}f'''(\xi_2) - 8\frac{h^3}{6}f'''(\xi_1)}{2h} = f'(x_j) + \frac{h^2}{3}(f'''(\xi_2) - 2f'''(\xi_1)) \tag{39}$$

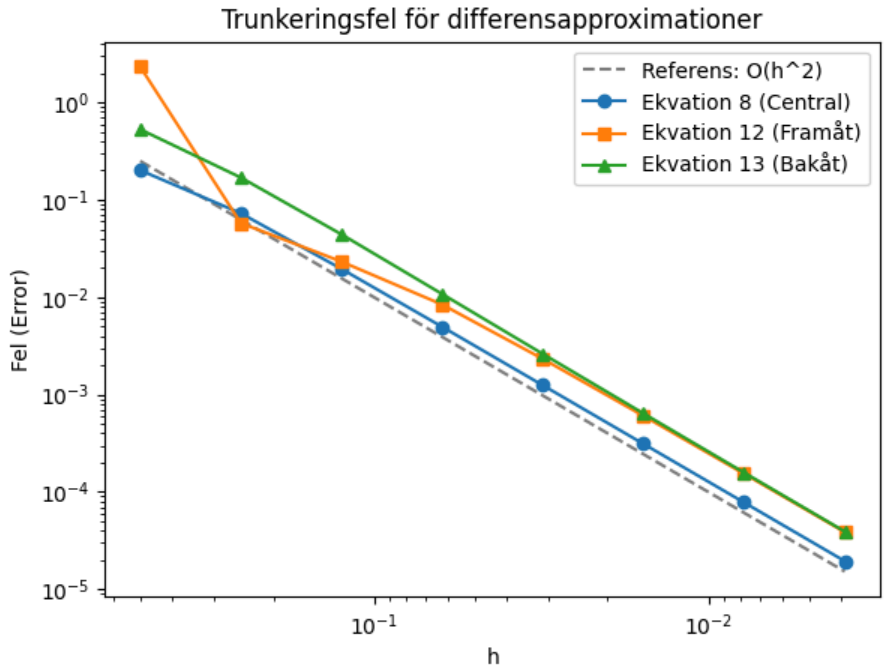
$$= f'(x_j) + \mathcal{O}(h^2) \tag{40}$$

Där $\xi_1, \xi_2 \in [x_j, x_j+2h]$. Detta visar att felet i approximationen är av ordning $\boldsymbol{h^2}$.

b) Denna uppgift skall implementera tre olika finita differnsmetoder i *Python* för att approximera derivatan av $\mathbf{sin}(\boldsymbol{e^x})$ vid $\boldsymbol{x = 0.75}$. Resultatet skall jämföras med den exakta derivatan, samt analysera plotta felet i varje metod för olika värden av $h = 2^{-k}$, där $k = 1, 2, 3, 4, 5, 6, 7, 8$. Beräkningarna gav följande resultat:

```
Analytical derivative at x=0.75: -1.09967e+00
h = 5.00000e-01, eq8: -1.30089e+00, eq12: -3.42609e+00, eq13: -5.70715e-01
h = 2.50000e-01, eq8: -1.17237e+00, eq12: -1.15732e+00, eq13: -9.30383e-01
h = 1.25000e-01, eq8: -1.11918e+00, eq12: -1.07648e+00, eq13: -1.05550e+00
h = 6.25000e-02, eq8: -1.10463e+00, eq12: -1.09127e+00, eq13: -1.08889e+00
h = 3.12500e-02, eq8: -1.10091e+00, eq12: -1.09734e+00, eq13: -1.09705e+00
h = 1.56250e-02, eq8: -1.09998e+00, eq12: -1.09907e+00, eq13: -1.09903e+00
h = 7.81250e-03, eq8: -1.09975e+00, eq12: -1.09952e+00, eq13: -1.09951e+00
h = 3.90625e-03, eq8: -1.09969e+00, eq12: -1.09963e+00, eq13: -1.09963e+00
```

,



Figur 5: Log log plot av felet för olika derivataapproximationer, vid olika värden av h

3.4 Randvärdesproblem del 2.2

3.4.1 Uppgift 7

Denna uppgift skall diskretisera intervallet $[0, L]$ i $N = 4$ delintervall, och approximera andra derivatan för Ekvation 14 med central differens, För att sätta upp det resulterande systemet av icke-linjära ekvationer och dess jacobimatris.

Det diskretiserade ntervallet $[0, L]$ ger 5 punkter: $x = 0,\ \frac{L}{4},\ \frac{L}{2},\ \frac{3L}{4},\ L$. Med dirichlet randvillkor, där $T(0) = T_s$ och $T(L) = T_\infty$, är de okända temperaturerna vid de inre punkterna $T_1 = T\left(\frac{L}{4}\right)$, $T_2 = T\left(\frac{L}{2}\right)$, och $T_3 = T\left(3\frac{L}{4}\right)$.

Från Ekvation 14 och Ekvation 16 kan uttrycket för varje inre punkt skrivas som:

$$F_j(T_{j-1}, T_j, T_{j+1}) = \frac{T_{j+1} - 2T_j + T_{j-1}}{h^2} - \alpha_1(T_j - T_\infty) - \alpha_2(T_j^4 - T_\infty^4) = 0 \tag{41}$$

$$\vec{F}(T_0, T_1, T_2) = (F_1, F_2, F_3) = \vec{0} \tag{42}$$

Detta system kan lösas med Newtons metod för system, där $\vec{T}^{k+1} = \vec{T}^k - J^{-1}\vec{F}(T^k)$. Jacobimatrisen består av de partiella derivatorna av varje F_j med avseende på T_{j-1}, T_j, och T_{j+1}.

$$J = \begin{bmatrix} \frac{\partial F_1}{\partial T_1} & \frac{\partial F_1}{\partial T_2} & \frac{\partial F_1}{\partial T_3} \\ \frac{\partial F_2}{\partial T_1} & \frac{\partial F_2}{\partial T_2} & \frac{\partial F_2}{\partial T_3} \\ \frac{\partial F_3}{\partial T_1} & \frac{\partial F_3}{\partial T_2} & \frac{\partial F_3}{\partial T_3} \end{bmatrix} \tag{43}$$

Jacobimatrisen kan delas upp i tre delar: den triangulära delen, den diagonala delen, och resten av matrisen, där varje del består av de partiella derivatorna av F_j med avseende på $T_{j\pm1}$, T_j, och T_{j+1} respektive.

$$\frac{\partial F_j}{\partial T_j} = -\frac{2}{h^2} - \alpha_1 - 4\alpha_2 T_j^3 \quad = \quad \frac{1}{h^2} \cdot (-k_j) \qquad \left(k_j = 2 + \alpha_1 h^2 + 4\alpha_2 h^2 T_j^3\right) \tag{44}$$

$$\frac{\partial F_j}{\partial T_{j-1}} = \frac{\partial F_j}{\partial T_{j+1}} = \frac{1}{h^2} \tag{45}$$

$$\frac{\partial F_j}{\partial T_{j+2}} = \frac{\partial F_j}{\partial T_{j-2}} = 0 \tag{46}$$

Detta ger Jacobimatrisen:

$$J = \frac{1}{h^2} \cdot \begin{bmatrix} -k_1 & 1 & 0 \\ 1 & -k_2 & 1 \\ 0 & 1 & -k_3 \end{bmatrix} \tag{47}$$

Newtons metod $J\Delta T = -\vec{F}(T_k) \quad (\Delta T = T_{k+1} - T_k)$ kan sedan lösas med hjälpa av Gauss-Jordans metod, eller `delta_T = np.linalg.solve(J, -F_vec)` i *Python*.

3.4.2 Uppgift 8.1

I Denna uppgift generaliseras uppgift 7 (3.4.1) för ett godtyckligt antal delintervall $\boldsymbol{N}$.

Enlig Ekvation 44, Ekvation 45, och Ekvation 46, kan Jacobimatrisen för ett godtyckligt antal delintervall N skrivas som en tridiagonal matris, där den diagonala delen består av elementen $-\frac{k_j}{h^2}$, och de två närmaste diagonalerna består av elementen $\frac{1}{h^2}$. Resten av matrisen består av nollor.

$$J = \frac{1}{h^2} \cdot \begin{bmatrix} -k_1 & 1 & 0 & \dots & 0 \\ 1 & -k_2 & 1 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 1 & -k_{N-2} & 1 \\ 0 & \dots & 0 & 1 & -k_{N-1} \end{bmatrix} \tag{48}$$

3.4.3 Uppgift 8.2

Denna uppgift skall implementera lösningen av randvärdesproblemet i Avsnitt 3.4.1 (Avsnitt 3.4.1) och Avsnitt 3.4.2 (Avsnitt 3.4.2) i *Python*.

```
def finite_difference(a_1, a_2, T_inf, T_s, L, N=100):
    x = np.linspace(0, L, N + 1) # Create a grid of points
    h = L / N # step size
    T0 = np.zeros(N+1) # Initialize an array to store the values at each point.
    T0[0] = T_s
    T0[-1] = T_inf

    def F(T): # Define the function F(T) that represents the system of equations
      F_vec = np.zeros(N-1)
      for i in range(1, N): # Calculate F_i using the finite difference approximation
```

```
        F_i = (T[i-1]-2*T[i]+T[i+1])-h**2*(a_1*(T[i]-T_inf)+a_2*(T[i]**4-T_inf**4))
        F_vec[i-1] = F_i
    F_list.append(F_vec)
    return F_vec

def jacobian(T): # Define the Jacobian matrix of F(T)
    J = np.zeros((N-1, N-1))
    for i in range(1, N):
        J[i-1, i-1] = -2-h**2*(a_1+a_2*4*T[i]**3) # Diagonal elements
        if i > 1: # Calculate the off-diagonal elements.
            J[i-1, i-2] = 1
        if i < N-2:
            J[i-1, i] = 1
    return J

def newton_method(T0, tol=1e-6, max_iter=1000):
    T = T0.copy()
    T0[i] = T_s + (T_inf - T_s) * (x[i] / L) # Linear interpolation start guess

    ''' code for newton method  can be found in previous sections '''

T = newton_method(T0)
return x, T
```

För att få en startgissning för T, kan en linjär interpolation mellan randvillkoren användas, där $T_i = T_s + (T_\infty - T_s) \cdot \left(\frac{x_i}{L}\right)$ för varje inre punkt x_i.

3.4.4 Uppgift 8.3

Denna uppgift skall använda den implementerade lösningen i uppgift 8.1 (3.4.3) för att lösa randvärdesproblemet för $N = 400$ delintervall, med givna parammtrar $T(L) = T_\infty$ $\alpha_1 = \frac{4h_c}{D \cdot K}$ och från Tabell 3. Lösningen skall sedan plottas och jämföras med den analytiska lösningen av det linjära problemet, och Noggranhetsordningen beräknas.

$$T(x) = T_\infty + (T_s - T_\infty) \cdot e^{-\sqrt{\alpha_1}x} \tag{49}$$

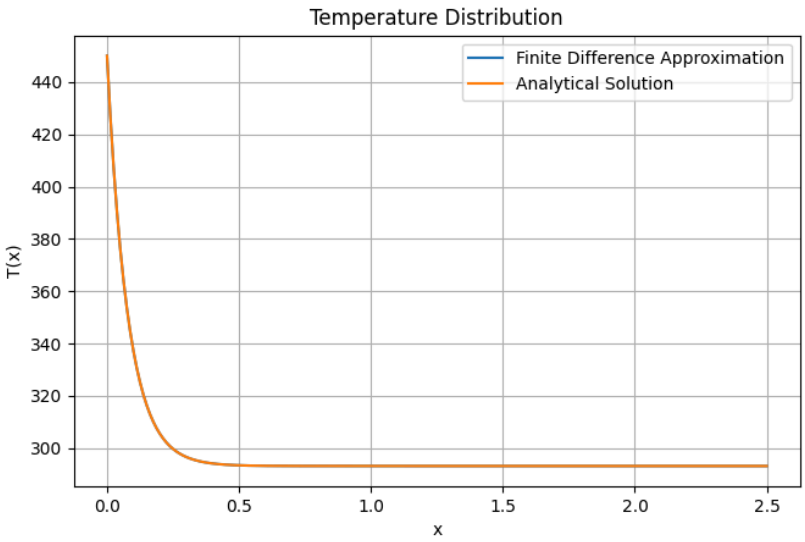
Parameter	Värde	Enhet
h_c	40	$W/(m^2 \cdot K)$
K	240	$W/(m \cdot K)$
ε	0.4	
D	$4.13 \cdot 10^{-3}$	m
T_s	450	K
T_∞	293	K

Tabell 3: Fläns parametervärden för att testa implementering.

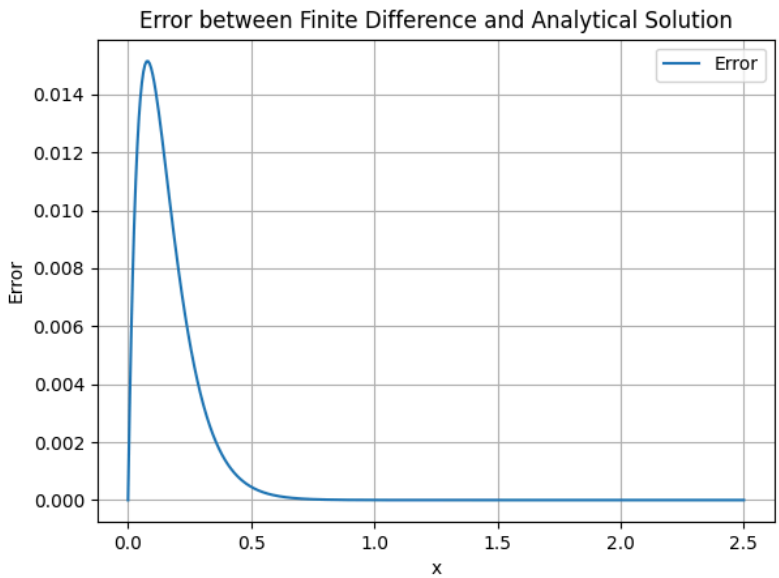
Den numeriska T lösningen kan utverderas mot den analytiska $\mathcal{T}$, med hjälp av root mean square error (RMSE) som mått på skillnaden mellan de två lösningarna.

$$e_{rms} = \left(\frac{1}{N-1} \cdot \sum_{k}^{N-1} (T_k - \mathcal{T}_k)^2 \right)^{1/2} \tag{50}$$

a) Den numeriska lösningen ger en mycket bra approximation av den analytiska lösningen, med ett fele på $\boldsymbol{e_{rms} = 0.00365\ K}$, som i plotten (Figur 6) ser identiska ut. Från Figur 7 kan det även ses att felet är störst nära $x = 0$ (se Avsnitt 4.0.4).



Figur 6: Numerisk lösning jämfört med analytisk lösning.

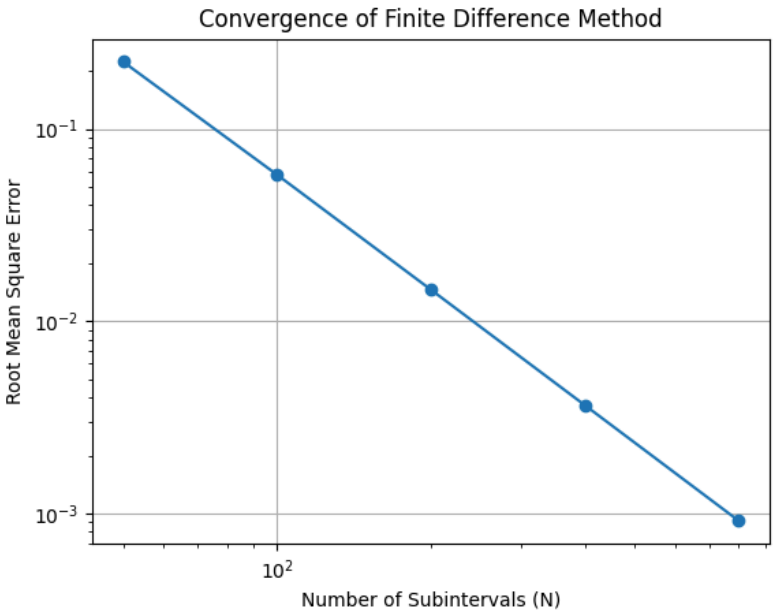


Figur 7: Skillnaden mellan numerisk och analytisk lösning över intervallet $[0, L](m)$.

b) För att beräkna norggrannhetsordningen, kan felet (rms Ekvation 50) beräknas för olika värden av $N = 50, 100, 200, 400, 800$ ($h = \frac{L}{N}$), och sedan använda formeln:

$$e \approx Ch^{-p} \Rightarrow \ln(e) = \ln(C) - p\ln(h) \Rightarrow p = \frac{\ln(e_2) - \ln(e_1)}{\ln(h_2) - \ln(h_1)} \tag{51}$$

Detta resulterade i en uppskattning av noggranhetsordningen $\boldsymbol{p = 1.98}$ (Figur 8), vilket är i linje med den teoretiska noggranhetsordningen för central differensmetoden, som är 2.



Figur 8: loglog plot av e mot n för att uppskatta noggranhetsordningen.

3.4.5 Del 8.4

Här ska både den linjära och icke-linjära termen i ekvationen Ekvation 14 lösas, för att modellera värmestrålningens effekt. Lösningen skall implementerar tre olika flänsar med olika materialegenskaper (Tabell 4), och resultaten jämföras in en plot.

givna paramtrar för systemet är: $L = 0.30m$, $D = 5.0 \cdot 10^{-3}m$, $,T(0) = 373.15K$, $T(L) = 293.15K$

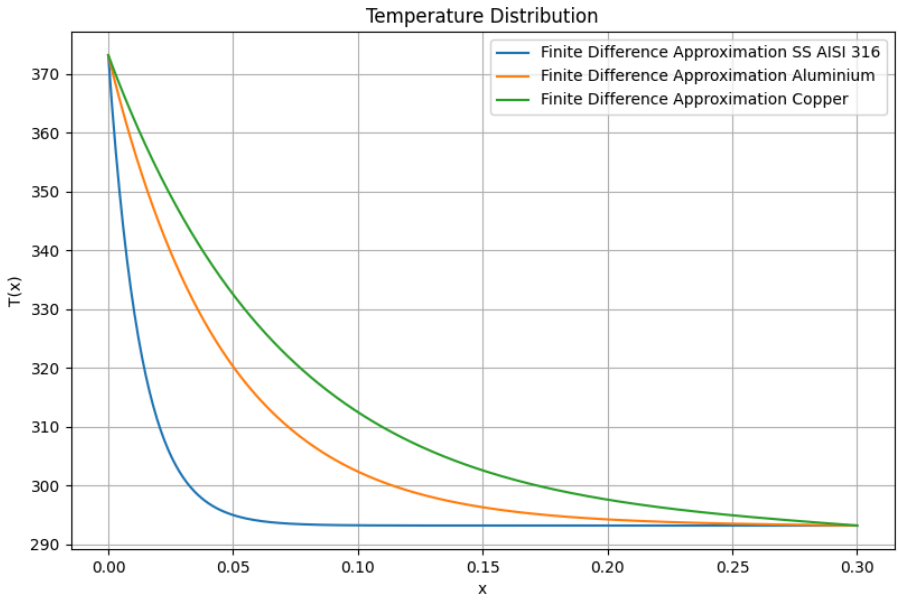
$$\alpha_1 = \frac{4h_c}{D \cdot K} \qquad \alpha_2 = \frac{4\varepsilon\sigma}{D \cdot K} \tag{52}$$

Där σ är Stefan-Boltzmanns konstant: $\sigma = 5.67 \cdot 10^{-8}\ W/(m^2 \cdot K^4)$

Parameter	**SS AISI 316**	**Aluminium**	**Koppar**	**Enhet**
K	14	180	398	$W/(m \cdot K)$
h_c	100	100	100	$W/(m^2 \cdot K)$
ε	0.17	0.82	0.03	

Tabell 4: Fläns parametervärden för olika konstruktioner.

a) Tempraturfördelningen i de tre olika materialen över Längden L kan ses i Figur 9, där det tydligt framgår att kopparflänsen har den mest effektiva värmeöverföringen, följt av aluminium och sedan SS AISI 316 (se Avsnitt 4.0.4).



Figur 9: Temperaturfördelning i de tre olika materialen över Längden L (x: meter, T: Kelvin).

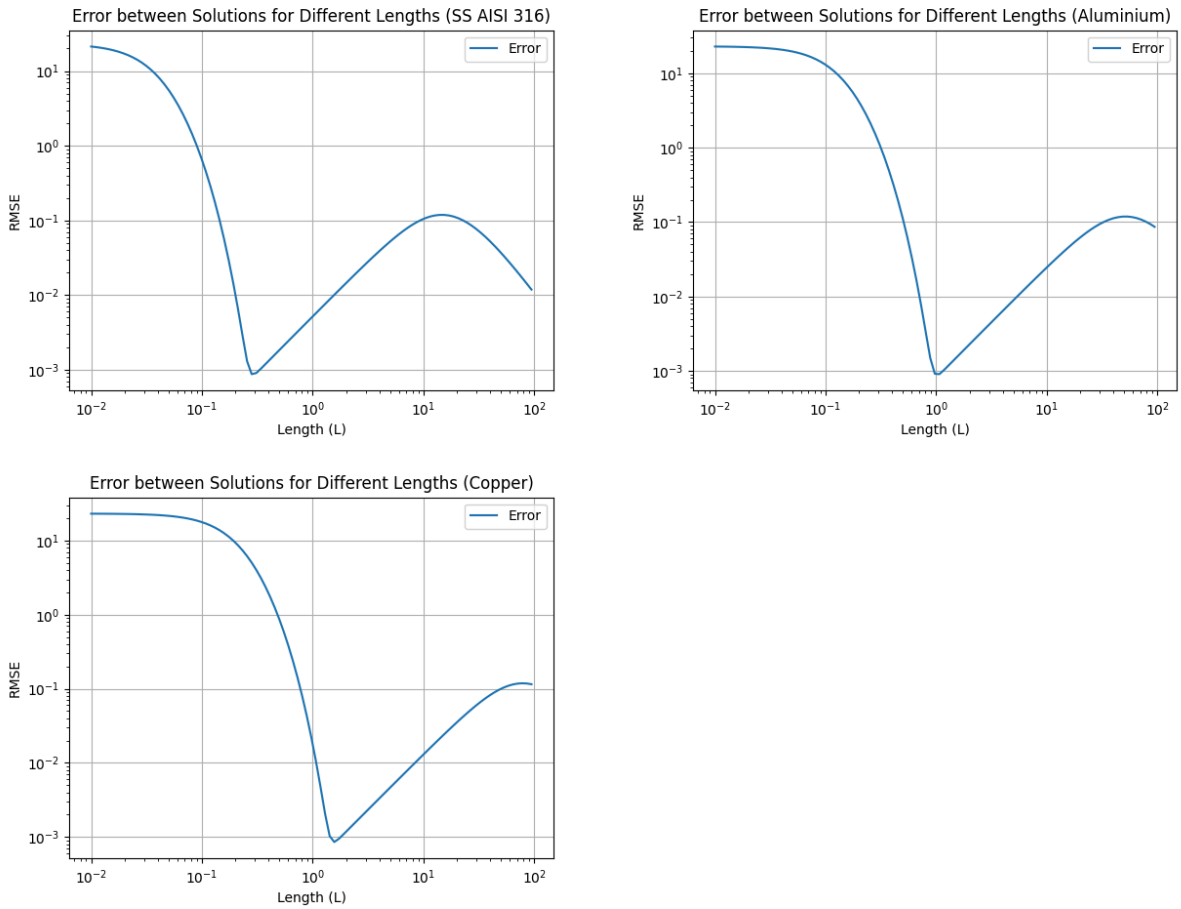
b) Genom att beräkna skillnaden i temperaturfördelningen (RMSE) för successivt ökand längder på flänsen (för de olika materialen), kan en längd på flänsen upskattas där skillnaden i temperaturfördelning mellan den nuvarande och tidigare längden mindre än en tolerans ($\|T_k - T_{k-1}\| < \varepsilon$). Detta ger en uppskattning av när värmestrålningens effekt kan ignoreras i designen av flänsar.

```
def compare_solutions(L1, L2, h=None):
    x1, T1 = finite_difference(a_1, a_2, T_inf, T_0, L1, N=400)
    x2, T2 = finite_difference(a_1, a_2, T_inf, T_0, L2, N=400)
    T2_on_x1 = np.interp(x1, x2, T2) # interpolate T2 onto the x1 grid
    error = T1 - T2_on_x1 # Calculate the error between the two solutions.
    rmse = np.sqrt(np.mean(error**2))
    return rmse

def find_infinite_length(L0=0.01, rel_tol=1e-3):
    L = L0
    max_L = 100.0  # Set a maximum length to prevent infinite loops
    error_list = []
    L_values = []
    tol = rel_tol * (T_0 - T_inf)  # Set the tolerance based on the relative error
    while L < max_L:
        err = compare_solutions(L, L*0.5)
        error_list.append(err)
        L_values.append(L)

        if err < tol:
            return L, error_list, L_values
        L *= 1.1
```

En rimlig uppskattning för längden där antagandet om oändlig längd skall vara nogrant bör därför vara **0.098m** för SS AISI 316, **0.340m** för aluminium, och **0.548m** för koppar, om toleransen sätts till $1\% = 0.80K$, och **0.159m** (SS AISI 316), **0.548m** (Aluminium) samt **0.882 m** (koppar) för en tolerans på $0.1\% = 0.080K$ (se @).



Figur 10: loglog plot av RMSE mellan temperaturfördelning för olika längder av flänsen, mot en fläns med 1.5 gånger längre längd, för de tre olika materialen.

4 Diskussion

4.0.1 Del 1.1 uppgift 3 C)

Att metoden ej konvergerar för startgissningen 1.5 beror sannolikt på att funktionens derivata är för låg vid denna punkt (Figur 6), vilket kan leda till att Newtons metod inte konvergerar.

En ytterdiameter på cirka 19 mm (som framtogs i Uppgift 2 3.1.2) är fortfarande en rimlig lösning för en värmeväxlare (Figur 1), och detta är därför den fysikaliskt relevanta roten i det aktuella intervallet. Det bör dock understrykas att den andra roten (kring 0.8314m) inte är en rimlig lösning för problemet, eftersom den ger en mycket stor ytterdiameter i förhållande till skalets innerdiameter $D_s = 1.219m$. Den fysikaliskt relevanta lösningen är därför roten nära $0.0191m$.

4.0.2 Del 1.2 uppgift 5

Utvärderingen av ekvationssystemet ger en ytterdiameter av rörest på 11.9mm, och en innerdiameter av skalet på 68.4cm. Dessa bedöms vara rimliga lösningar för konstruktionen av en värmeväxlare (Figur 1).

Att den uppskattade konvergensordningen för ekvationssystemet fluktuerar kraftigt under de första iterationerna är ett förväntat beteende (Figur 4). Detta beror på att Newtons metod endast är lokalt kvadratiskt konvergent. Under de första iterationerna befinner sig startgissningen utanför den region där Taylorutvecklingens andragradsterm dominerar, vilket gör att stegen styrs av funktionens globala icke-linjäritet.

4.0.3 Del 2.1 uppgift 6 b)

Att framåt- och bakåtdifferens ger ett något större fel än centraldifferens beror på trunkeringsfelets ledande konstant (Figur 5). Trots att alla tre metoder har noggrannhetsordning $\mathcal{O}(h^2)$, är konstanten för tre-punkts framåt- och bakåtdifferens $(\pm\frac{h^2}{3}f'''(\xi))$ ungefär dubbelt så stor som för centraldifferensen. $(-\frac{h^2}{6}f'''(\xi))$. I en log-log-plot resulterar denna faktor två i en parallellförskjutning uppåt.

Att felet för framåtdifferensen avviker från den raka linjen och ”dyker” för stora värden på h förklaras av att h inte är tillräckligt litet, varvid högre ordningars termer (som h^3 och h^4) i Taylorutvecklingen blir signifikanta. För specifika kombinationer av x och stora h kan feltermerna av olika ordning få motsatta tecken och därmed delvis ta ut varandra. Detta ger tillfälligt ett skenbart lägre fel innan metoden övergår i sitt förväntade asymptotiska beteende när h minskar.

4.0.4 Del 2.2

Uppgift 8.3 a Från Figur 7 kan det observeras att felet är störst nära $x = 0$. Detta är förväntat, eftersom det är där temperaturgradienten är som störst, vilket kan leda till större numeriska fel i den finita differens approximationen av andra derivatan. Även om felet är störst nära $x = 0$, så är det fortfarande mycket litet i absoluta termer, vilket indikerar att den numeriska lösningen är mycket noggrann över hela intervallet.

Uppgift 8.4 a) Tempraturfördelningen i de tre olika materialen över Längden L kan ses i Figur 9. kopparflänsen har den mest effektiva värmeöverföringen. Detta beror fysikaliskt på kopparns högre termiska konduktivitet, vilket möjliggör snabbare värmeöverföring längs flänsen.

b) För att praktiskt kunna jämföra skillnaden i temperaturfördelning för olika längder krävs det att lösningarna utvärderas vid samma x-punkter. Eftersom varje flänslängd ger ett eget diskretiserat rutnät, måste den ena lösningen därför interpoleras till det andra rutnätet innan RMSE beräknas.

Metoden hade möjligen också kunnat implementeras genom att jämföra temperaturfördelningen för en viss längd mot en mycket lång längd $\|T_{ref} - T_k\| < \varepsilon \quad (T_{ref} \approx T_\infty)$. Problemet med denna

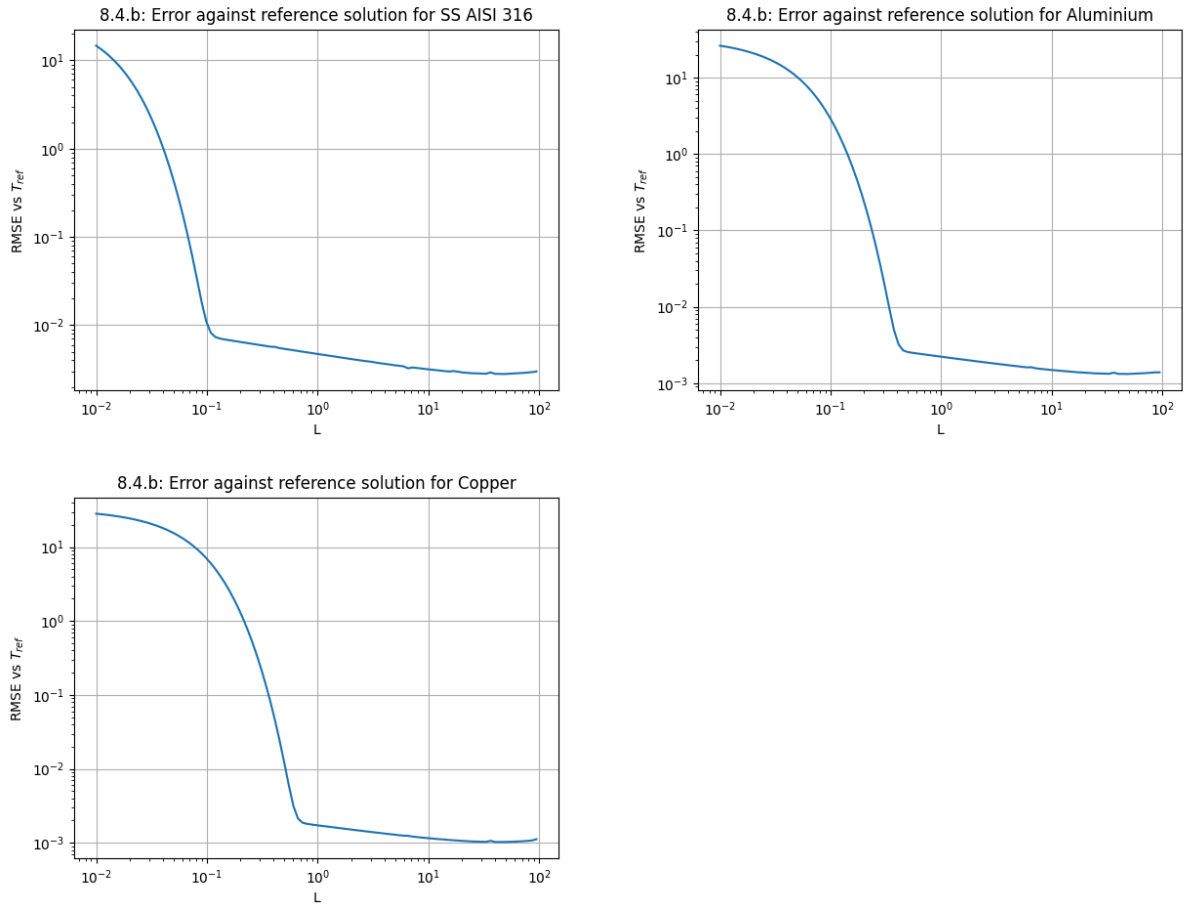
metod är att givet ett konstant antal delintervall $N = 400$, kommer steglängden h att öka. Detta medför att approximationen blir oprecis nära $x = 0$, där temperaturgradienten är som störst. Detta skulle kunna lösas genom att variera steglängden h för varje längd, beroende på gradienten.

Detta kunde implementeras separat och gav liknande uppskattningar av den kritiska längden där värmestrålningens effekt kan ignoreras: 0.074 m (SS AISI 316), 0.260 m (Aluminium) och 0.420 m (Koppar) för en tolerans på 0.1% (se Bilaga A 5.1).

Felet (Figur 10) avtar inte helt monotont, vilket sannolikt beror på att lösningarna jämförs på olika diskreta nät och att interpolation därför krävs. Eftersom ett fast antal delintervall används ökar dessutom steglängden när flänslängden växer, vilket påverkar noggrannheten nära $x = 0$, och kan ge små lokala variationer i felet.

5 Bilagor

5.1 Bilaga A



Figur 11: RMSE mellan temperaturfördelning för olika längder av flänsen, mot en referenslösning med mycket lång fläns (L=100m)

5.2 Bilaga B

```
Initial guess: d = 0.015000, D = 0.800000
Iteration 1: d = 0.014483, D = 0.753866, Error = 4.61e-02
Iteration 2: d = 0.013815, D = 0.736671, Error = 1.72e-02
Iteration 3: d = 0.013018, D = 0.715934, Error = 2.08e-02
Iteration 4: d = 0.012303, D = 0.696451, Error = 1.95e-02
Iteration 5: d = 0.011947, D = 0.686244, Error = 1.02e-02
Iteration 6: d = 0.011888, D = 0.684513, Error = 1.73e-03
Iteration 7: d = 0.011887, D = 0.684476, Error = 3.76e-05
Iteration 8: d = 0.011887, D = 0.684476, Error = 1.66e-08
Iteration 9: d = 0.011887, D = 0.684476, Error = 3.14e-15
```

Kodlistning 1: Iterativ lösning av Newtons metod för system, där både d och D_s löses ut samtidigt.